



Department of CSE

Mini Project Report - Task 3

“Malware Family Classification Using CNN Baselines and an Improved GCN on the MaleBin Dataset”

Course Code: ICE 478/CSE475

Section: 2

Track: Track 2 - GCN (Image-to-Graph)

Submitted to: Dr. Raihan Ul Islam, Associate Professor, Department of Computer Science and Engineering.

Submitted by: Group-1

Student ID	Student Name
2022-3-50-006	Md. Mustafidur Rahman
2022-3-50-008	Amana Akter Arfa
2022-3-50-025	Rakiba Alam

1. Introduction

Task 3 completes the Track 2 image-to-graph pipeline for malware family classification on the MaleBin dataset. The main goal was to improve the first PatchGCN from Task 2, determine which graph design choices actually help, compare the final GCN fairly with the strongest CNN baseline, test stability with five-fold stratified cross-validation, and examine model decisions using image and graph explainability methods. All experiments were carried out in Kaggle using PyTorch.

The Task 2 GCN represented each image using six handcrafted statistics per patch and achieved a macro-F1 of 0.3771. Task 3 replaces those low-dimensional handcrafted node attributes with learned CNN feature vectors before graph convolution. This change produced a much stronger image-to-graph representation while keeping the graph structure simple and interpretable.

2. Task 3 Objectives

- Improve the first GCN using stronger node features and graph/model tuning.
- Perform ablation experiments to measure the effect of graph connectivity and batch normalization.
- Compare the final GCN with the Task 2 first GCN and the strongest CNN baseline.
- Report complete multiclass metrics, ROC-AUC, precision-recall behavior, model size, and inference time.
- Run five-fold stratified cross-validation and statistical significance tests.
- Explain one correct and one wrong prediction using Grad-CAM, patch-level local surrogate importance, and GCN node importance.

3. Dataset Preparation and Data Integrity

The original MaleBin collection contained 12,464 grayscale malware images. Before splitting, SHA-256 hashes were computed to detect exact duplicates. Cross-class duplicate groups were removed and only one copy of exact within-class duplicates was retained. After duplicate control and retaining classes with at least 10 clean samples, 9,230 images across 35 malware families remained.

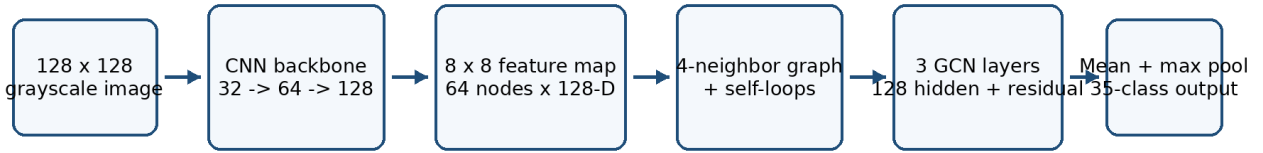
Item	Value
Original images	12,464
Cross-class duplicate groups removed	499
Usable images after cleaning	9,230
Classes after cleaning	35
Training split	6,461 (70%)
Validation split	1,384 (15%)
Test split	1,385 (15%)
Leakage check	PASS - duplicate hashes are disjoint across splits

Class imbalance was handled with a WeightedRandomSampler during training. Validation and test distributions were left unchanged. CNN inputs used 224 x 224 resolution, while graph-model inputs used 128 x 128 resolution. The split was stratified by class.

4. Improved Image-to-Graph Model

The strongest Task 3 change was the node representation. Instead of computing six pixel statistics from every patch, a small CNN backbone learns 128 feature channels and uses adaptive pooling to produce an 8 x 8 spatial feature map. Each spatial position becomes one graph node, so every image contains 64 nodes and every node has a learned 128-dimensional feature vector.

Improved Task 3 CNN-Feature GCN Pipeline



The main Task 3 change is the use of learned CNN feature vectors as graph-node attributes instead of six handcrafted patch statistics.

Figure 4.1: Improved CNN-feature GCN architecture used in Task 3

The selected graph connects each node to its horizontal and vertical neighbors and includes self-loops. The normalized adjacency matrix is used by three graph-convolution layers with 128 hidden features. Batch normalization and ReLU are applied after graph convolution, dropout is used for regularization, and a residual connection combines the last two GCN representations. Mean and maximum graph pooling are concatenated before the final 35-class classifier.

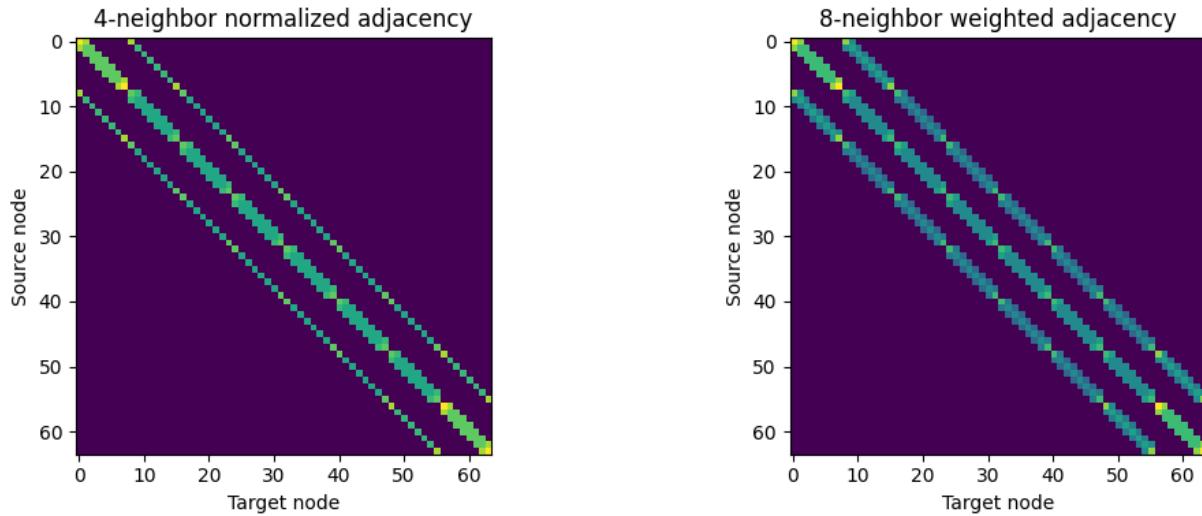


Figure 4.2: Normalized 4-neighbor and weighted 8-neighbor graph structures tested in the ablation study

4.1 Training Settings

Setting	Value
Optimizer	AdamW
Learning rate	0.001
Weight decay	0.0001
GCN hidden width	128
Dropout	0.30
GCN depth	3 layers
Fixed-split GCN epochs	5
Fixed-split ResNet18 epochs	6
CV epochs per fold	3
LR schedule	ReduceLROnPlateau
Early stopping	Enabled

Gradient clipping	1.0
-------------------	-----

5. Ablation Study

Four Task 3 variants were evaluated on the same held-out test split. The graph-free CNN backbone provides a direct test of whether message passing adds value. Two graph-connectivity schemes were compared, and batch normalization was removed in one 8-neighbor variant.

Model / Ablation	Accuracy	Balanced Acc.	Macro-F1	ROC-AUC	Macro AP
CNN only - graph off	0.7487	0.7571	0.7191	0.9737	0.7873
CNN + GCN, 4-neighbor	0.7884	0.8076	0.7857	0.9789	0.8316
CNN + GCN, weighted 8-neighbor	0.7762	0.7852	0.7645	0.9804	0.8211
CNN + GCN, 8-neighbor, no BN	0.7285	0.7554	0.7199	0.9661	0.7738

The 4-neighbor graph was the best GCN configuration with macro-F1 = 0.7857. Compared with the same CNN backbone without graph message passing, macro-F1 increased by 0.0666 (6.66 percentage points). The weighted 8-neighbor graph was slightly weaker, showing that adding more spatial connections did not automatically improve classification. Removing batch normalization from the 8-neighbor model reduced macro-F1 from 0.7645 to 0.7199, indicating that normalization helped stabilize the graph network.

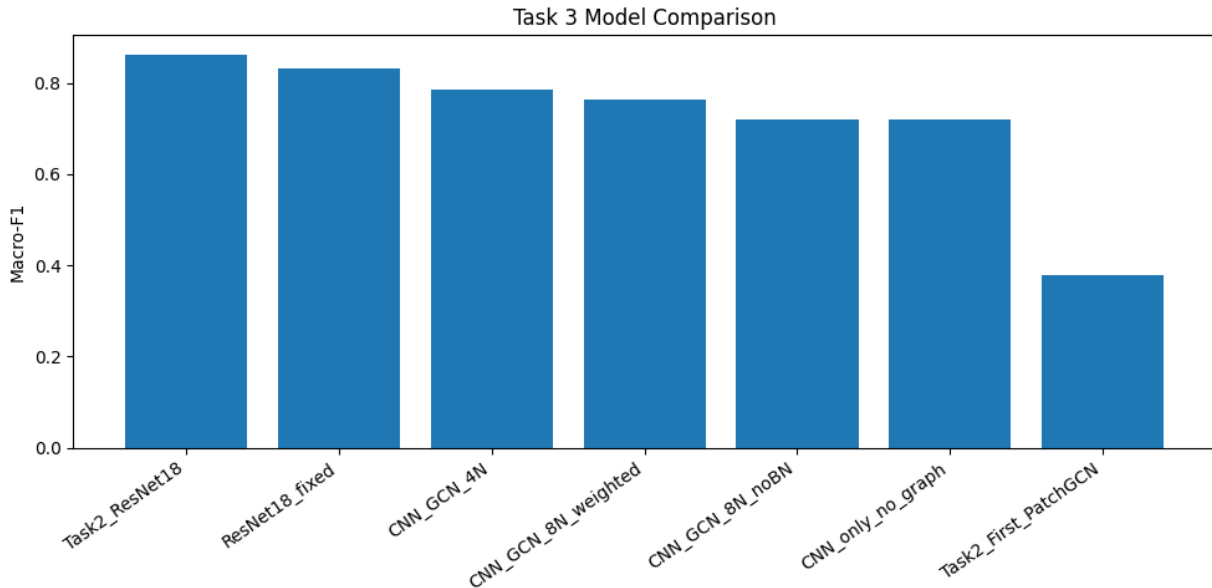


Figure 5.1: Macro-F1 comparison across Task 2 and Task 3 models

6. Final Model Performance

Model	Accuracy	Balanced Acc.	Macro-F1	Weighted-F1	Model Size	Inference
Task 2 ResNet18	0.8664	0.8811	0.8615	0.8648	-	-
Task 2 first PatchGCN	0.4433	0.5080	0.3771	0.3887	-	-
Task 3 ResNet18 (paired run)	0.8347	0.8519	0.8323	0.8365	42.76 MB	1.39 ms/image
Task 3 final CNN_GCN_4N	0.7884	0.8076	0.7857	0.7857	0.61 MB	1.20 ms/image

The improved GCN raised macro-F1 from 0.3771 in the first PatchGCN to 0.7857, an absolute gain of 0.4086. The strongest CNN remained better on the fixed test split, but the final GCN became much more competitive

while using a dramatically smaller checkpoint. The paired Task 3 ResNet18 checkpoint was used for significance testing because both models produced predictions on the same test samples.

6.1 Per-Class Behavior

Several families were classified almost perfectly, including Adialer.C, Agent.FYI, Alueron.gen!J, Dialplatform.B, Fakerean, Lolyda.AT, Obfuscator.AD, Rbot!gen, VB.AT, and Wintrim.BX. Difficult classes included CoinMinerXMRig ($F1 = 0.233$), Swizzor.gen!I (0.174), Remcos (0.308), Gh0stRAT (0.361), and Trickbot (0.407). These lower class-level scores explain why macro-F1 remains below overall accuracy.

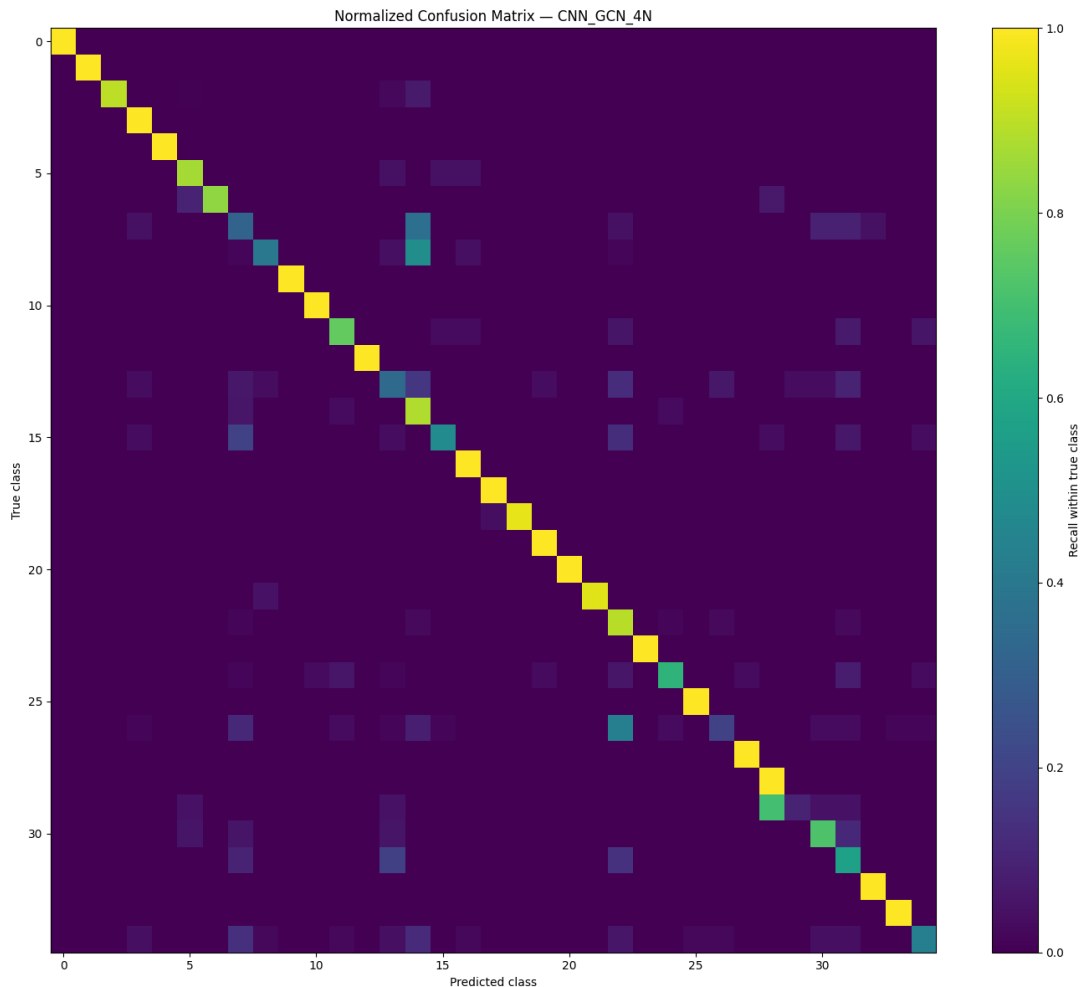


Figure 6.1: Normalized confusion matrix of the final CNN_GCN_4N model

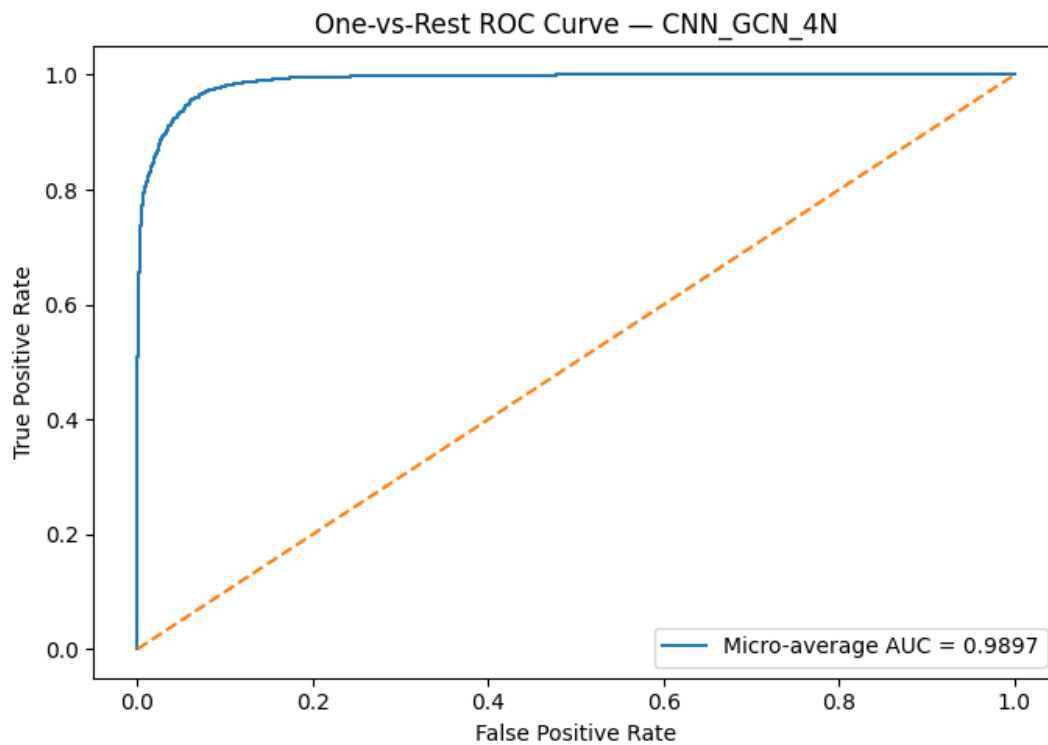


Figure 6.2: One-vs-rest ROC curve for the final GCN

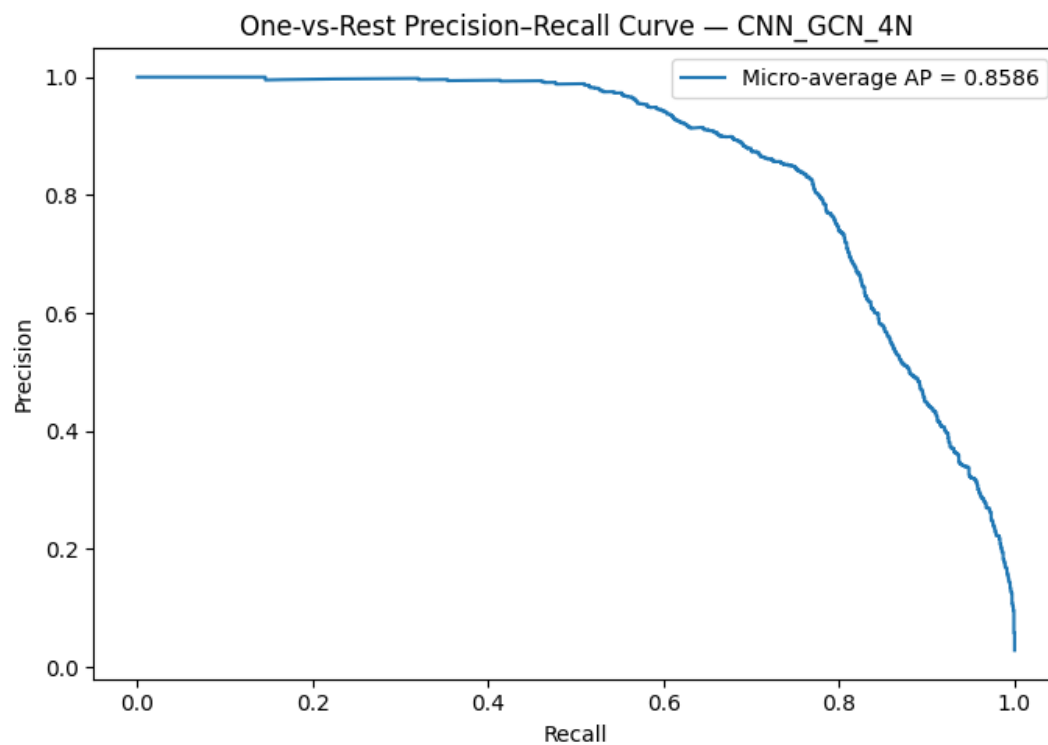


Figure 6.3: One-vs-rest precision-recall curve for the final GCN

The final model achieved macro one-vs-rest ROC-AUC = 0.9789 and macro average precision = 0.8316. The micro-average ROC curve reached AUC = 0.9897, while the micro-average precision-recall AP was 0.8586. These curves show strong ranking ability even though several minority classes remain difficult.

7. Five-Fold Cross-Validation

Five-fold stratified cross-validation was used to evaluate stability. The same fold assignments were used for ResNet18 and the final GCN. Because the full deep-learning experiment is computationally expensive, each fold used the reduced three-epoch training schedule defined in the notebook.

Fold	ResNet18 Macro-F1	Final GCN Macro-F1
1	0.6507	0.6673
2	0.7358	0.6748
3	0.7684	0.6119
4	0.7768	0.6986
5	0.7230	0.7617

Metric	ResNet18 Mean +/- SD	Final GCN Mean +/- SD
Accuracy	0.7278 +/- 0.0735	0.7304 +/- 0.0295
Balanced accuracy	0.7654 +/- 0.0499	0.7289 +/- 0.0437
Macro-F1	0.7309 +/- 0.0501	0.6829 +/- 0.0543
Weighted-F1	0.7212 +/- 0.0741	0.7103 +/- 0.0347
Macro ROC-AUC	0.9782 +/- 0.0047	0.9725 +/- 0.0068
Macro average precision	0.8249 +/- 0.0165	0.7862 +/- 0.0326

The final GCN produced slightly higher mean accuracy and noticeably lower accuracy variation, but ResNet18 produced the higher mean balanced accuracy, macro-F1, ROC-AUC, and average precision. Therefore, the cross-validation result does not support a claim that the final GCN consistently outperforms the CNN baseline.

8. Statistical Significance

Two significance tests were reported because both a shared held-out test set and five paired cross-validation scores were available.

Test	Comparison	Statistic / Counts	p-value	Conclusion at alpha = 0.05
Exact McNemar	Task 3 ResNet18 vs final GCN	b = 128, c = 64	0.000004	Significant
Wilcoxon signed-rank	5-fold Macro-F1	W = 3.0	0.3125	Not significant

McNemar testing on the same 1,385 test samples found significantly more cases where ResNet18 was correct and the final GCN was wrong than the reverse. This supports the fixed-test advantage of ResNet18. In contrast, the Wilcoxon test across only five fold-level macro-F1 values was not significant ($p = 0.3125$), so the cross-validation evidence does not establish a reliable fold-wise difference at $\alpha = 0.05$.

9. Explainability Analysis

Explainability was examined with three complementary views: Grad-CAM for the ResNet18 baseline, patch-level local surrogate importance for the final GCN, and GCN node-occlusion importance. One highly confident correct prediction and one highly confident error were selected automatically from the test set.

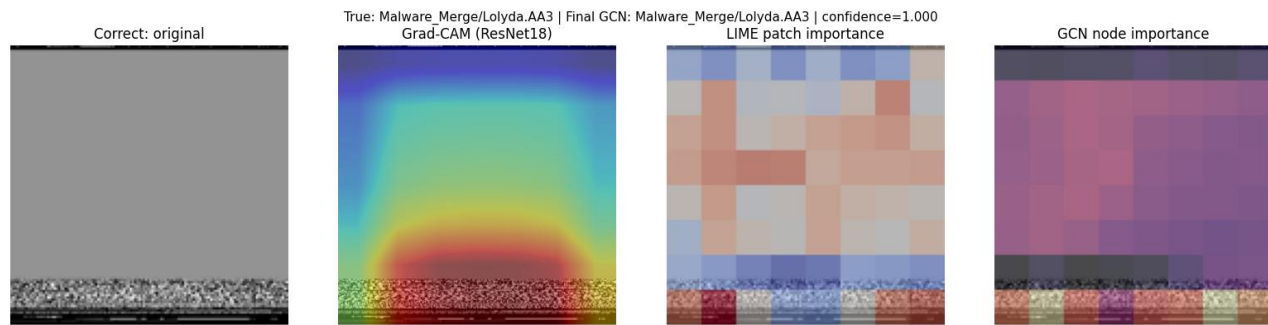


Figure 9.1: Correct prediction - Lolyda.AA3 classified as Lolyda.AA3 (GCN confidence 0.9998)

For the correct Lolyda.AA3 example, Grad-CAM emphasizes the lower structured band of the byte image. The patch-level and graph-node explanations also assign noticeable importance to lower patches and nearby central regions. The agreement is not exact because the explanation methods operate on different representations, but all three indicate that the decision is driven by localized texture patterns rather than the uniform upper area.

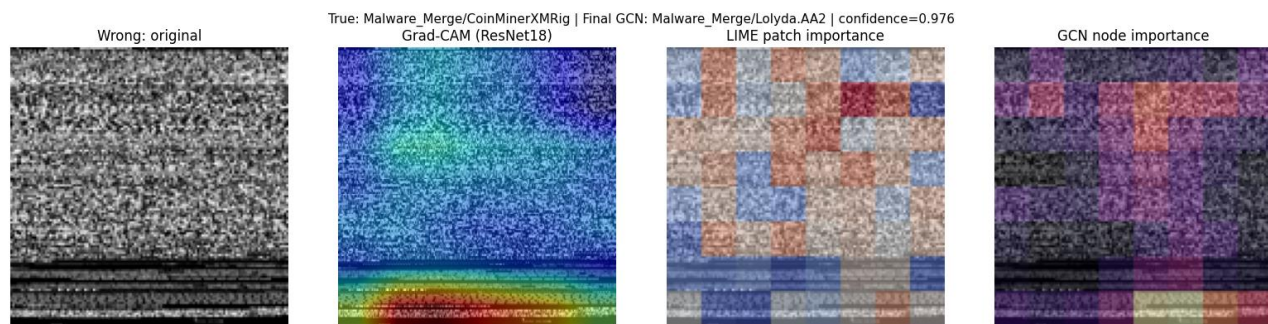


Figure 9.2: Wrong prediction - CoinMinerXMRig classified as Lolyda.AA2 (GCN confidence 0.9759)

For the misclassified CoinMinerXMRig example, the explanation is more distributed. Grad-CAM strongly responds to the lower horizontal structure, while patch-level and node-occlusion maps highlight several central and lower patches. The model is therefore highly confident even though it focuses on texture patterns that are not sufficiently specific to the true family.

Interpretation limitation: MaleBin images are generated from malware bytes rather than natural physical scenes. A highlighted patch indicates a region that affected the image classifier, but it should not be interpreted as direct evidence of a specific malware function, code block, or causal program behavior.

10. Comparison with Related Work

The five Task 1 studies were conducted on different malware datasets and used different experimental protocols and model families. For that reason, their published results are not treated as directly comparable numerical baselines for the MaleBin split used here. The comparison is limited to methodological context, which avoids claiming a numerical win across incompatible datasets.

Study	Dataset	Method	Directly comparable?
Self-Supervised Vision Transformers for Malware Detection (2022)	AndroZoo	Vision Transformer	No
Grayscale Image-Based Deep Learning Model for Efficient and Scalable Malware Detection (2025)	MalImg	CNN	No
Multimodal Malware Classification Using Proposed Ensemble DNN Framework (2025)	Android malware dataset	Ensemble DNN	No
YoloMal-XAI (2025)	CICMalDroid2020	YOLO11 + XAI	No
Hybrid Noise-Resistant Technique for Malware Classification (2026)	BODMAS	CAE + LBP + XGBoost	No

11. Reproducibility and Practical Limitations

The final architecture selected in both Task 3 notebooks was CNN_GCN_4N. The explainability notebook loaded the saved Kaggle version of that architecture. Because the saved Kaggle version re-executed GPU training, its recorded fixed-test score differed slightly from the interactive run used for the main comparison table. This small run-to-run variation does not change the selected architecture or the main conclusion. The report uses one internally consistent set of fixed-test results from the improvement-and-ablation notebook and uses the saved checkpoint only for the explainability examples.

The experiment also uses only three epochs per cross-validation fold to keep the full five-fold deep-learning evaluation practical. Longer training and more extensive hyperparameter search could improve both models. Finally, the regular patch graph is a convenient spatial construction, but malware byte images do not contain natural semantic regions, so a GCN is not guaranteed to outperform a conventional CNN.

12. Conclusion

Task 3 substantially improved the graph-based classifier. Replacing handcrafted patch statistics with learned CNN feature nodes increased macro-F1 from 0.3771 for the Task 2 first PatchGCN to 0.7857 for the selected 4-neighbor CNN-feature GCN. Ablation showed that graph message passing improved the CNN-only backbone, while denser 8-neighbor connectivity did not help and batch normalization was beneficial. The final GCN remained below ResNet18 on the fixed test split, and McNemar testing confirmed that difference, while the five-fold Wilcoxon test did not find a significant fold-wise difference. The final model is therefore best described as a much stronger and compact graph-based alternative rather than a replacement for the strongest CNN. Grad-CAM, local patch importance, and graph-node importance provided useful model-sensitivity views, with the required caution that byte-image regions do not have direct malware-code semantics.

References

- [1] S. Seneviratne, R. Shariffdeen, S. Rasnayaka, and N. Kasthuriarachchi, "Self-Supervised Vision Transformers for Malware Detection," IEEE Access, 2022. <https://ieeexplore.ieee.org/iel7/6287639/9668973/09889730.pdf>
- [2] "Grayscale Image-Based Deep Learning Model for Efficient and Scalable Malware Detection," 2025. <https://ieeexplore.ieee.org/document/11448123>
- [3] "Multimodal Malware Classification Using Proposed Ensemble Deep Neural Network Framework," Scientific Reports, 2025. <https://www.nature.com/articles/s41598-025-96203-3.pdf>
- [4] "YoloMal-XAI: Interpretable Android Malware Classification Using RGB Images and YOLO11," Journal of Cybersecurity and Privacy, 2025. <https://www.mdpi.com/2624-800X/5/3/52/pdf>
- [5] S. S. H. Shah, A. Hamed, A. U. R. Khan, and M. Nicho, "Hybrid Noise-Resistant Technique for Malware Classification," Scientific Reports, 2026. <https://www.nature.com/articles/s41598-026-57376-7.pdf>